GKGaaS Graph Engine

JSON DSL → SQL compiler

Why not just write SQL?

- **Agent Friendly** - LLMs generate structured JSON reliably
- **Security** - No string concatenation, no injection
- **Portability** - Backend can change without breaking clients
- **Easy to Sync** - Schema derived from ontology

```
{
  "query_type": "traversal",
  "nodes": [
    { "id": "u", "entity": "User",
      "columns": ["username", "name"], "node_ids": [1]
},
    { "id": "mr", "entity": "MergeRequest",
      "columns": ["iid", "title", "state",
"source_branch"] },
    { "id": "p", "entity": "Project",
      "columns": ["name", "full_path"] }
  ],
  "relationships": [
    { "type": "AUTHORED", "from": "u", "to": "mr" },
    { "type": "IN_PROJECT", "from": "mr", "to": "p" }
  ],
  "limit": 50
}
```

How do we implement this?

JSON DSL Query

```
{
  "query_type": "traversal",
  "nodes": [
    { "id": "u", "entity": "User",
      "columns": ["username"],
      "node_ids": [1] },
    { "id": "mr", "entity": "MergeRequest",
      "columns": ["title"] },
    { "id": "p", "entity": "Project",
      "columns": ["name"] }
  ],
  "relationships": [
    { "type": "AUTHORED",
      "from": "u", "to": "mr" },
    { "type": "IN_PROJECT",
      "from": "mr", "to": "p" }
  ]
}
```

→ **compiler** →

Generated SQL

```
SELECT u.username, mr.title, p.name
FROM gl_user AS u
JOIN gl_edges AS e0
  ON u.id = e0.source_id
JOIN gl_merge_request AS mr
  ON e0.target_id = mr.id
JOIN gl_edges AS e1
  ON mr.id = e1.source_id
JOIN gl_project AS p
  ON e1.target_id = p.id
WHERE u.id IN ({p0:Int64})
  AND e0.relationship_kind = {type_e0:String}
  AND e1.relationship_kind = {type_e1:String}
  AND startsWith(mr.traversal_path, {p1:String})
  AND startsWith(p.traversal_path, {p1:String})
```

The Compiler

```
fn compile(json: &str, ontology: &Ontology, ctx: &SecurityContext) -> Result<SQL> {
    let value = validate_json(json)?;            // JSON structure ok?
    validate_ontology(&value, ontology)?;        // entities exist?
    let input = parse(value)?;                   // JSON → typed struct
    validate(&input, ontology)?;                 // references valid?
    let input = normalize(input, ontology);      // canonicalize
    let mut ast = lower(&input)?;                // build SQL AST
    let ctx = enforce_return(&mut ast, &input)?; // for redaction
    apply_security(&mut ast, ctx)?;              // tenant isolation
    codegen(&ast, ctx)                           // AST → SQL
}
```

Phase 1: Schema Validation

schema.json

```
{
  "required": ["query_type"],
  "properties": {
    "query_type": {
      "enum": ["traversal", "search",
               "aggregation", "path_finding"]
    },
    "nodes": {
      "type": "array",
      "items": { "$ref": "#/$defs/NodeSelector" }
    },
    "limit": {
      "type": "integer",
      "minimum": 1,
      "maximum": 1000
    }
  },
  "allOf": [{
    "if": { "query_type": "traversal" },
    "then": { "nodes": { "minItems": 2 } }
  }]
}
```

+

JSON DSL Query

```
{
  "query_type": "traversal",
  "nodes": [
    { "id": "u", "entity": "User" },
    { "id": "mr", "entity": "MR" }
  ],
  "limit": 50
}
```

↗

Error:

"traversal" requires "nodes" minItems: 2

↘

Accept

Phase 2: Ontology Validation + Parse

```rust
fn compile(json: &str, ontology: &Ontology, ctx: &SecurityContext) -> Result<SQL> {
    let value = validate_json(json)?;            // JSON structure ok?
    validate_ontology(&value, ontology)?;        // entities exist?
    let input = parse(value)?;                   // JSON → typed struct
    validate(&input, ontology)?;                 // references valid?
    let input = normalize(input, ontology);      // canonicalize
    let mut ast = lower(&input)?;                // build SQL AST
    let ctx = enforce_return(&mut ast, &input)?; // for redaction
    apply_security(&mut ast, ctx)?;              // tenant isolation
    codegen(&ast, ctx)                           // AST → SQL
}
```

Phase 2: Ontology Validation + Parse (Cont.)

schema.json

```
{
  "$defs": {
    "EntityType": { "enum": [] },
    "RelationshipTypeName": { "enum": [] },
    "NodeProperties": {}
  }
}
```

+

ontology/core/nodes/project.yaml

```
node_type: Project
properties:
  id: { type: int64 }
  name: { type: string }
```

→

Validate Populated JSON Schema

↗ **Error**

↘ **parse()**

At runtime: enum: [] becomes enum: ["User", "Project", "MergeRequest", ...]

Phase 3: Semantic Validation

```rust
fn compile(json: &str, ontology: &Ontology, ctx: &SecurityContext) -> Result<SQL> {
    let value = validate_json(json)?;            // JSON structure ok?
    validate_ontology(&value, ontology)?;        // entities exist?
    let input = parse(value)?;                   // JSON → typed struct
    validate(&input, ontology)?;                 // references valid?
    let input = normalize(input, ontology);      // canonicalize
    let mut ast = lower(&input)?;                // build SQL AST
    let ctx = enforce_return(&mut ast, &input)?; // for redaction
    apply_security(&mut ast, ctx)?;              // tenant isolation
    codegen(&ast, ctx)                           // AST → SQL
}
```

- Nodes exist and have entity types
- Relationship endpoints point to declared nodes
- Column names match the entity's schema
- Aggregation targets are real nodes
- Order by references valid node + property
- Goal: fold this into jsonschema so we have one validation pass

Phase 4: Normalize

```rust
pub fn compile(json_input: &str, ontology: &Ontology, ctx: &SecurityContext) -> Result<ParameterizedQuery> {
    let value = validate_json(json_input)?;
    validate_ontology(&value, ontology)?;
    let input: Input = serde_json::from_value(value)?;
    validate::validate(&input, ontology)?;
    let input = normalize::normalize(input, ontology);
    let mut node = lower::lower(&input)?;
    let result_context = enforce_return(&mut node, &input)?;
    apply_security_context(&mut node, ctx)?;
    codegen(&node, result_context)
}
```

Transforms:

- `"entity": "User"` → `"table": "gl_user"`
- `"columns": "*"` → `["id", "username", "email", ...]`
- Enum integers → string labels

Phase 5: Lower

```rust
fn compile(json: &str, ontology: &Ontology, ctx: &SecurityContext) -> Result<SQL> {
    let value = validate_json(json)?;            // JSON structure ok?
    validate_ontology(&value, ontology)?;        // entities exist?
    let input = parse(value)?;                   // JSON → typed struct
    validate(&input, ontology)?;                 // references valid?
    let input = normalize(input, ontology);      // canonicalize
    let mut ast = lower(&input)?;                // build SQL AST
    let ctx = enforce_return(&mut ast, &input)?; // for redaction
    apply_security(&mut ast, ctx)?;              // tenant isolation
    codegen(&ast, ctx)                           // AST → SQL
}
```

Phase 5: Lower (Cont.)

Validated Input → AST →

```
Query {
  select: vec![
    SelectExpr::new(Expr::col("p", "path_ids"), "path_ids"),
    SelectExpr::new(
      Expr::func("arrayConcat", vec![
        Expr::col("p", "path"),
        Expr::func("array", vec![next_tuple])
      ]), "path"),
  ],
  from: TableRef::join(
    JoinType::Inner,
    TableRef::scan("paths", "p"),
    TableRef::scan("gl_edges", "e"),
    Expr::eq(Expr::col("p", "node_id"), Expr::col("e", "source_id"))
  ),
  where_clause: Expr::and(
    Expr::lt(Expr::col("p", "depth"), Expr::param("max_depth")),
    Expr::not(Expr::func("has", vec![...]))
  ),
  ..Default::default()
}
```

Scary!

It’s just a query builder

Prisma

```
prisma.user.findMany({
where: { email: { contains: "@" } },
include: { posts: true }
})
```

Drizzle

```
db.select()
.from(users)
.leftJoin(posts, eq(id, authorId))
.where(like(email, '%@%'))
```

Us

```
Query {
select: vec![...],
from: TableRef::join(...),
where_clause: Expr::and(...),
}
```

Same idea: build a tree, emit SQL

Phase 5: Lower (Cont.) - Query Types

```
pub fn lower(input: &Input) -> Result<Node> {
    match input.query_type {
        QueryType::Traversal | QueryType::Search => lower_traversal(input),
        QueryType::Aggregation => lower_aggregation(input),
        QueryType::PathFinding => lower_path_finding(input),
        QueryType::Neighbors => lower_neighbors(input),
    }
}
```

Each query type has its own lowering strategy:

- **Traversal/Search** - JOIN chain
- **Aggregation** - GROUP BY
- **PathFinding** - Recursive CTE
- **Neighbors** - Edge table scan

Phase 6: Enforce Return Context for Redaction

```
fn compile(json: &str, ontology: &Ontology, ctx: &SecurityContext) -> Result<SQL> {
    let value = validate_json(json)?;            // JSON structure ok?
    validate_ontology(&value, ontology)?;        // entities exist?
    let input = parse(value)?;                   // JSON → typed struct
    validate(&input, ontology)?;                 // references valid?
    let input = normalize(input, ontology);      // canonicalize
    let mut ast = lower(&input)?;                // build SQL AST
    let ctx = enforce_return(&mut ast, &input)?; // for redaction
    apply_security(&mut ast, ctx)?;              // tenant isolation
    codegen(&ast, ctx)                           // AST → SQL
}
```

- For each entity alias (u, mr, p), inject hidden columns: `_gkg_u_id`, `_gkg_u_type`, etc.
- Type column is a literal: `'User' AS _gkg_u_type`, id is the actual value from the database
- Server extracts a list of tuples like `[(102, "User"), (103, "Project")]` to perform redaction against
- Special code path in the server for `neighbors` / `path_finding` to perform redaction against the dynamic nodes
- **This works because the query is an AST - we can manipulate it with code!**

Phase 7: Security Context Injection

```rust
fn compile(json: &str, ontology: &Ontology, ctx: &SecurityContext) -> Result<SQL> {
    let value = validate_json(json)?;            // JSON structure ok?
    validate_ontology(&value, ontology)?;        // entities exist?
    let input = parse(value)?;                   // JSON → typed struct
    validate(&input, ontology)?;                 // references valid?
    let input = normalize(input, ontology);      // canonicalize
    let mut ast = lower(&input)?;                // build SQL AST
    let ctx = enforce_return(&mut ast, &input)?; // for redaction
    apply_security(&mut ast, ctx)?;              // tenant isolation
    codegen(&ast, ctx)                           // AST → SQL
}
```

- Walk AST, find all table scans (skip edge table, skip `gl_users`)
- Inject `startsWith(traversal_path, $MY_TRAVERSAL_PATH)` into WHERE clause for each table
- Multiple paths? Use `startsWith(LOWEST_COMMON_PREFIX) AND (p1 OR p2 OR ...)`

Phase 8: Codegen

```
fn compile(json: &str, ontology: &Ontology, ctx: &SecurityContext) -> Result<SQL> {
    let value = validate_json(json)?;            // JSON structure ok?
    validate_ontology(&value, ontology)?;        // entities exist?
    let input = parse(value)?;                   // JSON → typed struct
    validate(&input, ontology)?;                 // references valid?
    let input = normalize(input, ontology);      // canonicalize
    let mut ast = lower(&input)?;                // build SQL AST
    let ctx = enforce_return(&mut ast, &input)?; // for redaction
    apply_security(&mut ast, ctx)?;              // tenant isolation
    codegen(&ast, ctx)                           // AST → SQL
}
```

Phase 8: Codegen (Cont.)

AST

```
Query {
  select: vec![
    SelectExpr::new(
      Expr::col("u", "name"),
      "u_name"
    ),
  ],
  from: TableRef::scan(
    "gl_user", "u"
  ),
  where_clause: Expr::eq(
    Expr::col("u", "id"),
    Expr::param("p0")
  ),
  limit: Some(10),
}
```

→

codegen()

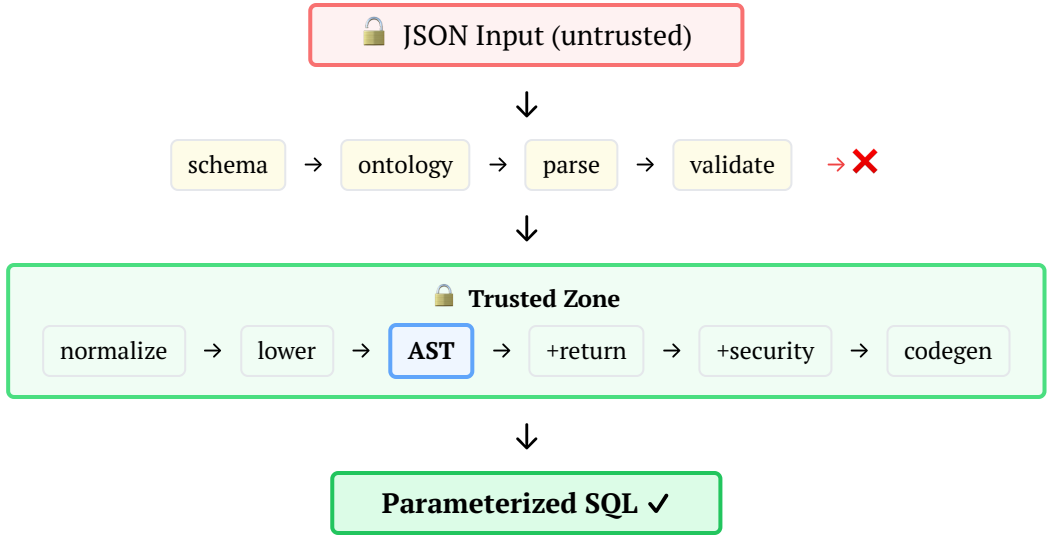
```
let select_items: Vec<_> = q.select
    .iter()
    .map(|sel| {
        let expr = self.emit_expr(&sel.expr);
        match &sel.alias {
            Some(a) => format!("{expr} AS {a}"),
            None => expr,
        }
    }).collect();
parts.push(format!("SELECT {}",
    select_items.join(", ")));
parts.push(format!("FROM {}", from.sql));
if !where_parts.is_empty() {
    parts.push(format!("WHERE {}",
        where_parts.join(" AND ")));
}
```

→

Parameterized SQL

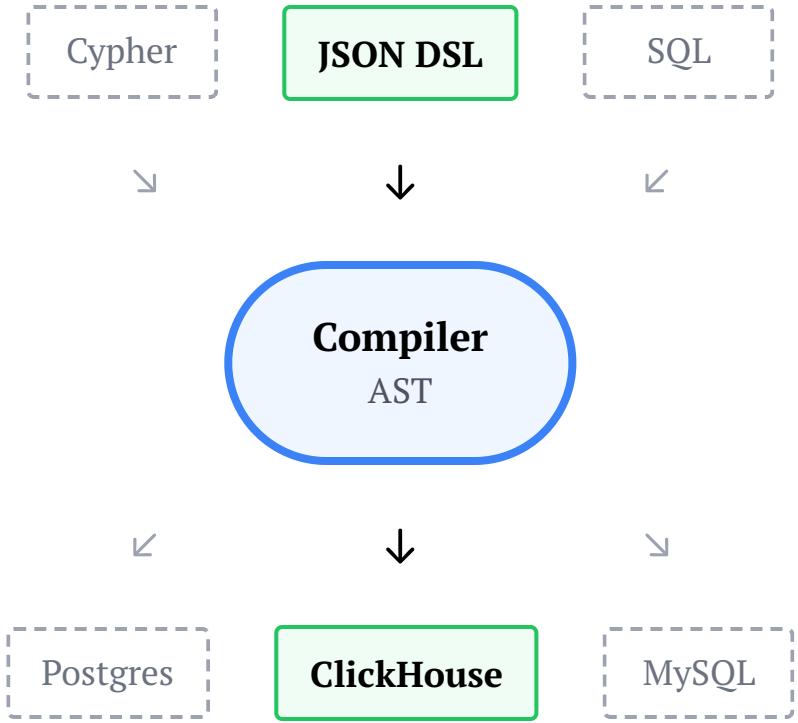
```
SELECT
  u.name AS u_name,
  u.id AS _gkg_u_id,
  'User' AS _gkg_u_type
FROM gl_user AS u
WHERE u.id = {p0:Int64}
  AND startsWith(
    u.traversal_path, '1/2/')
LIMIT 10
```

Summary

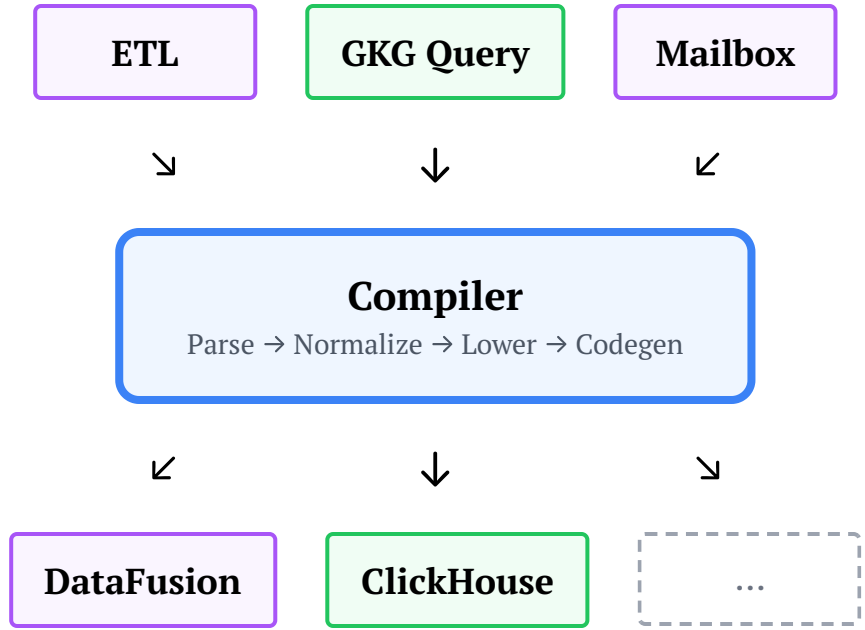


- Validate early, fail fast - bad queries should never touch the database
- Strong separation of concerns - lowering should never involve validation, etc.
- Security policies + redaction context injected via AST manipulation
- Parameterized SQL output - hardened

Vision


Cypher
JSON DSL
SQL
Compiler
AST
Postgres
ClickHouse
MySQL

Vision (cont.)


ETL
GKG Query
Mailbox
Compiler
Parse → Normalize → Lower → Codegen
DataFusion
ClickHouse
...

Readiness Questions

- **Secure Aggregates**:
  - Do we need row-level redaction for aggregated entities? Or are Reporter+/Planner+ roles sufficient?
- **Approximate Aggregates**: Could bounded random sampling (e.g., sample for AVG) solve redaction limitations?
  - For large datasets?
- **Security Threshold**: What threat modeling is sufficient for launch?
  - Beyond Sec reviews, what tools are used to validate ~similar~ access patterns to GKG like GraphQL? Fuzzing?
- **Performance**: Defining acceptable performance standards for GKG queries
  - What is acceptable latency, memory utilization per query for .com, self-managed?
- **Entity Rollout**: 6 domains (core, code_review, ci, security, plan, source_code) with 25+ entity types.
  - More rigorous integration testing per entity?
- **SSOT**: Ontology-driven schema validation. Interlock w/ Jean-Gabriel
  - How do we keep ontology in sync with Rails models?

Thank You!

Appendix

Redaction Flow

Query Engine provides metadata for post-query redaction validation.

Query Engine (`crates/query-engine`)

```
// result_context.rs - Metadata for redaction
pub struct RedactionNode {
    pub alias: String,
    pub entity_type: String,
    pub id_column: String,   // _gkg_{alias}_id
    pub type_column: String, // _gkg_{alias}_type
}

// return.rs - Inject columns into SELECT
fn enforce_return_columns(q: &mut Query, ...) {
    for node in &input.nodes {
        ctx.add_node(&node.id, entity);
        q.select.push(SelectExpr {
            expr: Expr::col(&node.id, "id"),
            alias: Some(id_column(&node.id)),
        });
    }
}
```

Query Pipeline (`crates/gkg-server`)

```
// service.rs - Pipeline orchestration
let compiled = compile(query_json, &ontology, &ctx)?;
let batches = self.execute_query(&compiled).await?;

let extracted = self.extraction.execute(batches);
let authorized = AuthorizationStage::execute(extracted).aw
let redacted = RedactionStage::execute(authorized);

// redaction.rs - Apply authorizations
input.query_result.apply_authorizations(
    &input.authorizations,
    &entity_map
);
```

Redaction Flow - Schema Introspection

For `path_finding` and `neighbors` queries, entity types are unknown at compile time.

Dynamic Node Discovery

```
// query_result.rs - Extract from Arrow columns
fn extract_path_nodes(batch, row_idx) -> Vec<NodeRef> {
    // _gkg_path: Array(Tuple(Int64, String))
    let col = batch.column(PATH_COLUMN);
    // Returns [(node_id, entity_type), ...]
}

fn extract_neighbor_node(batch, row_idx) -> Option<NodeRef
    // _gkg_neighbor_id: Int64
    // _gkg_neighbor_type: String (runtime value!)
    let id = batch.column(NEIGHBOR_ID_COLUMN);
    let entity = batch.column(NEIGHBOR_TYPE_COLUMN);
    Some(NodeRef::new(id, entity))
}
```

Why Not ResultContext?

```
// ResultContext works for known entities:
ctx.add_node("u", "User");  // User is known
ctx.add_node("p", "Project"); // Project is known

// But path_finding/neighbors discover types at runtime:
// - Path: User -> Project -> Group -> ???
// - Neighbors: User's neighbors could be ANY type

// Solution: store dynamic_nodes per row
struct QueryResultRow {
    columns: HashMap<String, ColumnValue>,
    dynamic_nodes: Vec<NodeRef>, // runtime discovery
    authorized: bool,
}
```

General Hydration Model

Move from query-specific hydration to a unified post-redaction enrichment pattern.

Current: Neighbors-only Hydration

```rust
// hydration.rs - Only for neighbors queries
pub async fn execute(&self, mut result: QueryResult, ...)
    if !matches!(result_context.query_type,
                 Some(QueryType::Neighbors)) {
        return Ok(result);  // Skip non-neighbors
    }

    let refs = self.extract_entity_refs(&result);
    let props = self.fetch_all_properties(&refs).await?;
    self.merge_properties(&mut result, &props);
    Ok(result)
}
```

Future: Universal Hydration

```rust
// Generalized pattern for all query types
pub async fn execute(&self, result: QueryResult, ...) {
    // 1. Get authorized entity refs (already redacted)
    let refs = result.authorized_rows()
        .flat_map(|r| r.all_node_refs())
        .collect();

    // 2. Batch fetch properties by type
    let props = self.fetch_properties_batch(&refs).await?;

    // 3. Hydrate (works for any query type)
    self.merge_properties(&mut result, &props);
    Ok(result)
}
```

Benefits: Decouple query execution from property fetching. Query returns IDs + edges, hydration fetches full entities. Enables caching, batching, and field selection.

Performance

Simulator Run — 29 queries, 11M nodes + 100M edges, ~954 MiB memory limit

Metric	Value
Success rate	89.7% (26/29)
Total time	4,614 ms
Mean	**159 ms**
Median	**26 ms**
Min	8 ms
Max	1,389 ms

Slowest Queries	Time
High weight work items	1,389 ms
Confidential work items	84 ms
Critical vulnerabilities	66 ms
Pipeline success rate	61 ms
MR reviewers per project	53 ms

3 failures (MEMORY_LIMIT_EXCEEDED): Shortest path CTE, Vulnerabilities→MRs join, Long-running pipelines

Simulator

Generate synthetic graphs and evaluate query performance at scale.

ClickHouse Configuration

```yaml
# simulator.yaml
clickhouse:
  url: http://localhost:8123
  database: default

  schema:
    engine: MergeTree
    index_granularity: 8192
    node_primary_key: [traversal_path, id]
    edge_order_by: [source_id, source_kind,
                    relationship_kind]

    projections:
      - name: by_target
        table: edges
        order_by: [target_id, target_kind]
```

Graph Generation Config

```yaml
generation:
  seed: 42
  organizations: 1

  roots:
    User: 10000    # 10k users
    Group: 20      # 20 top-level groups

  relationships:
    CONTAINS:
      "Group -> Project": 10
    IN_PROJECT:
      "MergeRequest -> Project": 100
      "Pipeline -> Project": 100

  associations:
    AUTHORED:
      "User -> MergeRequest": 100  # 30M edges
```

Workflow: `generate` → `load` → `evaluate` (100M edge scale test)

Appendix: Probabilistic Aggregates

Problem: Redaction filters rows *after* aggregation → leaks counts of inaccessible data

Solution: Sample + redact *before* aggregation

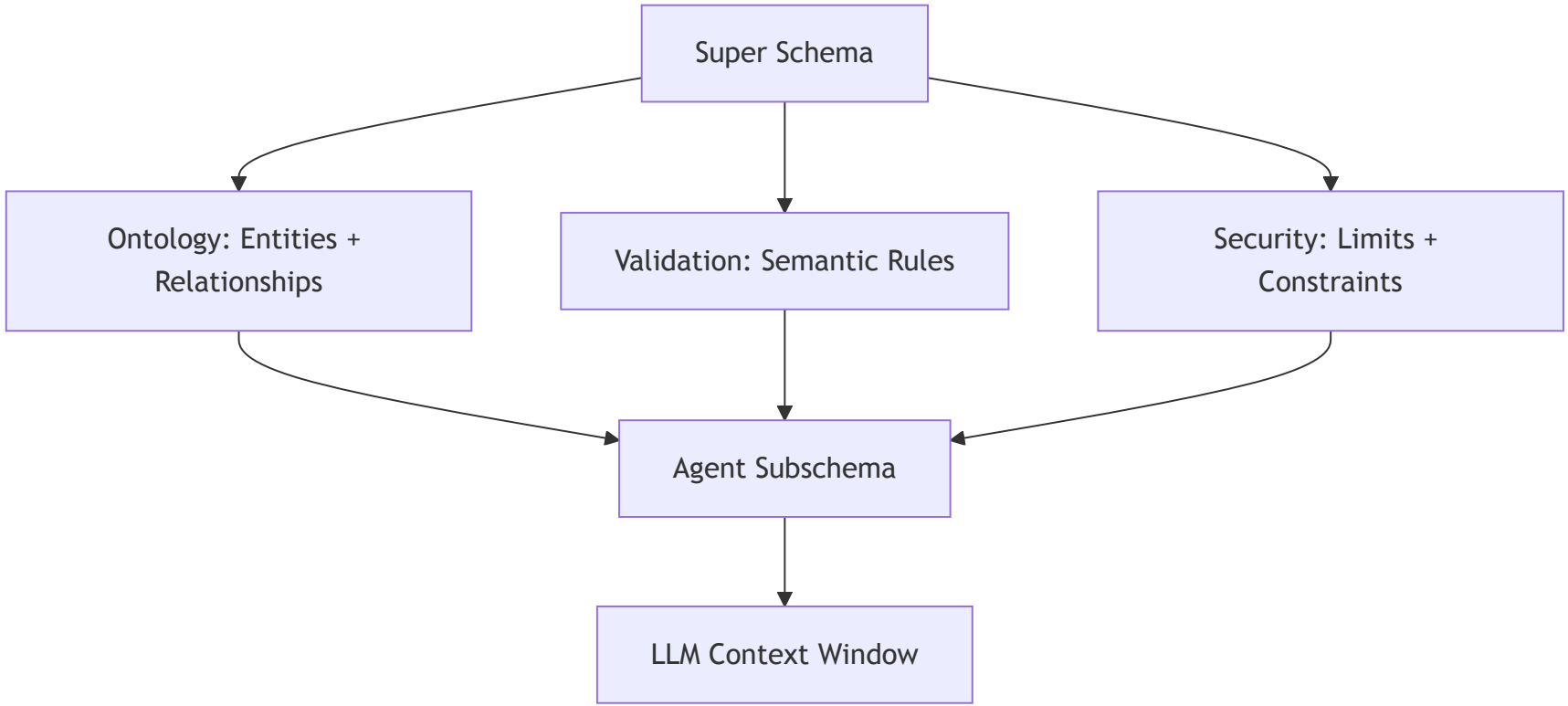
Current (leaky)

```sql
SELECT u.id, COUNT(mr.id) as mr_count
FROM users u
JOIN edges e ON ...
JOIN merge_requests mr ON ...
GROUP BY u.id
-- Redaction happens AFTER count
```

Proposed (secure)

```sql
SELECT u.id,
  COUNT(mr.id) * (1.0 / 0.1) as mr_count_approx
FROM users u
JOIN edges e ON ...
JOIN merge_requests mr ON ...
WHERE cityHash64(mr.id) % 100 < 10  -- 10% sample
  AND mr.id IN (accessible_ids)      -- redact first
GROUP BY u.id
```

- Bounded random sample (e.g., 10%) before redaction gives ~90% confidence interval
- Statistical guarantees vs exact counts - acceptable for dashboards/analytics
- ClickHouse `cityHash64` provides deterministic, uniform sampling

Appendix: JSON Schema Architecture

Goal: Single source of truth schema that powers validation, agents, and documentation